

Introducción a PHP

Luis Valencia Cabrera (lvalencia@us.es),
Víctor Ramos González (vramos1@us.es)

Departamento de Ciencias de la Computación e Inteligencia Artificial
Universidad de Sevilla

12-04-2023, Bases de Datos

Introducción

- Comunicación cliente - servidor
- PHP (acrónimo de “PHP: Hypertext Preprocessor”) es un lenguaje interpretado de alto nivel embebido en páginas HTML y ejecutado en el servidor.
- Su nombre es una broma, ya que la primera P de las siglas hace referencia a PHP. Es un nombre *recursivo*.

Introducción

Comunicación entre el cliente y el servidor **sin** PHP:

- Escribimos en la barra del navegador la dirección y el archivo a solicitar.
- El navegador envía el mensaje a través de Internet a otro ordenador, por ejemplo <http://www.cs.us.es/cursos/bd/Welcome.html> solicitando la página (archivo) `Welcome.html`.
- El servidor web (que puede ser el Apache, IIS, etc.), programa que se ejecuta en la máquina `www.cs.us.es`, recibe el mensaje y lee el archivo solicitado desde el disco duro.
- El servidor web envía el archivo solicitado por el navegador tal cual está en el disco duro.
- El navegador muestra en pantalla el archivo que envió el servidor web.

Introducción

Comunicación entre el cliente y el servidor **con** PHP:

- Escribimos en el navegador la dirección del archivo a solicitar.
- El navegador envía el mensaje a través de Internet a otro ordenador solicitando la página `fichero.php`.
- El servidor recibe el mensaje y al ver que la extensión es “php” solicita al intérprete de PHP (otro programa que se ejecuta en el servidor web) que ejecute el archivo y devuelva la respuesta.
- El intérprete PHP lee desde el disco el archivo `fichero.php`.
- El intérprete PHP **ejecuta los comandos contenidos en el archivo (y eventualmente se comunica con un gestor de base de datos)** (por ejemplo MySQL).

Comunicación entre el cliente y el servidor **con** PHP:

- Tras ejecutar el programa contenido en el archivo, envía su respuesta al servidor web.
- Éste envía la respuesta al cliente que la había solicitado.
- El navegador muestra en pantalla el archivo que envió el servidor web.
- En PHP la salida puede ser la página HTML de respuesta (ésta viaja por internet y llega al cliente que la solicitó), o bien un archivo JSON, XML, etc.

Introducción

- Para agregar un programa PHP dentro de una página HTML debemos por un lado al crear el archivo definirlo con extensión php (a diferencia de las páginas estáticas que tienen extensión html) y dentro del contenido de la página, encerrar el programa entre los símbolos

```
<?php /* Mi programa PHP */ ?>
```

- El programa podrá ejecutar instrucciones que pueden actuar sobre archivos, bases de datos, etc., y además podrá ir emitiendo una salida a quien lo llamó. Esta salida podrá ser un texto, un código HTML, un objeto JSON u otros.
- Dentro del mismo archivo php, podrá haber muchos fragmentos de código encerrados entre los símbolos indicados arriba.

Un primer ejemplo

Veamos este primer ejemplo:

```
<html>
<head></head>
<body>
  <?php
    echo "Hola Mundo";
  ?>
</body>
</html>
```

En PHP generamos salida con comandos **echo** o **print**. Además, todo lo que queda fuera de `<?php` y `?>` va directo a la salida también. Si llamamos a la página desde el navegador, este recibirá el código HTML generado y lo interpretará.

Introducción

Vamos a probar este primer ejemplo:

- Creamos una carpeta PHP en C://xampp/htdocs/bd2022 (o la ruta que corresponda, donde tengamos los archivos alojados en nuestro servidor web).
- Guardamos el fichero ej01.php en la carpeta creada.
- Iniciamos el servidor (Apache) desde XAMPP
- Para ver la salida del programa, escribimos en el navegador:
localhost/bd2022/PHP/ej1.php

Comentarios

Comentarios dentro de `<?php y ?>`

- *// Esto es un comentario en una línea*
- *# Esto también es un comentario en una línea*
- */* Esto es un comentario
de más de una línea */*

Comentarios fuera de `<?php y ?>`, siempre que la salida a generar sea HTML:

- *<!-- Comentario de HTML -->*

En el ejemplo `ej02.php` tenemos algo más de explicación sobre estos comentarios.

Variables

- Las **variables** empiezan siempre con el símbolo de dólar (\$).
- La asignación de valores a variables emplea el signo igual (=).
- Las variables definidas fuera de las funciones son globales.
Para acceder desde una función a una variable global, se debe indicar dentro de la función: **global \$a, \$b;**
- PHP distingue entre mayúsculas y minúsculas.
- Como podemos ver en [este enlace](#), existen funciones como **print_r** o **var_dump** para ver el contenido de una variable, con algunas diferencias en el grado de detalle sobre el mismo.

Ejemplo

```
<?php
    $dia1=15;
    $dia2=30;
    echo "El valor de la primera variable es ";
    echo $dia1;
    echo "<br>";
    echo "El valor de la segunda variable es ";
    echo $dia2;
?>
```

Este ejercicio, ej03.php, declara y usa variables, imprimiendo su valor. En el ejercicio ej04.php se continúan usando la asignación y la impresión pero añadiendo explicaciones adicionales.

Prueba el siguiente fichero

```
<!DOCTYPE html>
<html>
<body>
    <?php
        $color = "azul";
        echo "Mi coche es " . $color . "<br>";
        echo "Mi camisa es " . $COLOR . "<br>";
        echo "Mi cuaderno es " . $coLoR . "<br>";
    ?>
</body>
</html>
```

¿Qué ha observado a la salida de este ejemplo, ej05.php? ¿Qué le sugiere lo que ve? (No se preocupe por los puntos que verá en las instrucciones, en un momento veremos el papel que hacen.)

Otro ejemplo

- `$dia = date("d");` guarda en la variable `$dia` el valor del día del mes con dos dígitos (con ceros iniciales).
- `$dia = date("j");` guarda en la variable `$dia` el valor del día del mes sin ceros iniciales
- `$mes = date("n");` guarda en la variable `$mes` una representación numérica del mes, sin ceros iniciales.
- `$year = date("Y");` guarda en la variable `$year` el año actual con 4 dígitos.
- `$ds=date("w");` guarda en la variable `$ds` una representación numérica del día de la semana (0 para domingo, 1 lunes...)
- Podemos encontrar más formas de dar la fecha [aquí](#).

Tipos de variables

- Como habrá podido observar en estos primeros ejemplos, y como ocurriría en JS, no ha sido necesario declarar el tipo de las variables empleadas.
- No obstante, existen muchos tipos de datos en PHP, como podemos revisar [aquí](#).
- Uno de los tipos más habituales, las cadenas de caracteres, se pueden escribir entre comillas simples o dobles. Podemos concatenar cadenas con el contenido de expresiones usando el operador de concatenación, el punto (.)
- Podemos hacer [interpolación de variables](#) dentro de una cadena siempre que usemos comillas dobles.
- Además de estos tipos de datos básicos hay muchos otros, como vimos en el ejemplo de fechas.

Concatenación e interpolación de cadenas

```
<?php
    $dia = date("j");
    $mes = date("n");
    $year = date("Y");
    echo "Hoy es " . $dia . " del mes " .
        $mes . " de " . $year;
    echo "<br>";
    echo "Hoy es $dia del mes $mes de $year";
?>
```

En este ejemplo, ej06.php, se emplea la concatenación de cadenas, usando el operador punto (.), y a continuación se repite el proceso empleando interpolación de variables dentro de la cadena, con comillas dobles "Texto \$var texto".

Algunos tipos de datos

La función var_dump()

<?php

```
$x1 = "Hello world!";
```

```
$x2 = 5985;
```

```
$x3 = 10.365;
```

```
$x4 = true;
```

```
var_dump($x1);
```

```
var_dump($x2);
```

```
var_dump($x3);
```

```
var_dump($x4);
```

ej8.php:9:string 'Hello world!' (Length=12)

ej8.php:10:int 5985

ej8.php:11:float 10.365

ej8.php:12:boolean true

?>

En este ejemplo (ej07.php) empleamos var_dump para mostrar tanto el tipo de datos como el contenido de las variables.

Algunos tipos de datos

La función print_r()

```
<?php
```

```
$x1 = "Hello world!";
```

```
$x2 = 5985;
```

```
$x3 = 10.365;
```

```
$x4 = true;
```

```
print_r($x1);
```

```
echo "<br>";
```

```
print_r($x2);
```

```
echo "<br>";
```

```
print_r($x3);
```

Hello world!

5985

10.365

1

En este otro ejemplo (ej08.php) vemos que print_r solamente muestra el contenido, no el tipo. Lo emplearemos sobre todo para ver el contenido de un array asociativo.

Tipos de dato array

```
<?php  
$coches = array("Volvo", "BMW", "Toyota");  
echo "<br>";  
echo "<br>";  
echo "La posici&ocute;n 0 es $coches[0] <br>";  
echo "La posici&ocute;n 1 es $coches[1] <br>";  
echo "La posici&ocute;n 2 es $coches[2] <br>";  
?>
```

En este primer ejemplo sobre arrays (ej09.php) se declara un array ("clásico", del tipo de los que estamos acostumbrados a recorrer en JavaScript), \$coches, y se accede a sus posiciones (a través de sus índices, comenzando en 0).

Tipos de dato array

```
<?php
```

```
$dia = date("j");  
$ds = date("w");  
$dias_cast = array(  
    "domingo", "lunes",  
    "martes", "miércoles", "jueves",  
    "viernes", "sábado"  
);  
echo "Hoy es " . $dias_cast[$ds] . " " . $dia;
```

```
?>
```

Este nuevo ejemplo (ej10.php) hace básicamente lo mismo, ya que `$ds` contiene un número que indica el día de la semana (empezando por el 0, que indica el domingo), `$dias_cast` contiene un día de la semana en cada posición (empezando también por la 0 - domingo), de modo que accedemos al array por posición usando `$dias_cast[$ds]`.

Tipos de dato array

Arrays asociativos

```
<?php
$o = array(
    "a" => 4,
    "b" => "perro",
    "c" => FALSE,
    "d" => array(
        "x" => 2.3, "y" => 3.4
    ),
    "e" => array("pan", "leche", "patatas")
);
echo "Mi $o[b] tiene $o[a] a~tilde;os";
echo " y tiene la posi~n (" .
    $o["d"]["x"] . ", " . $o["d"]["y"] . ")";
```

En este otro ejemplo (ej11.php) vemos cómo declarar arrays asociativos (similar a objetos JSON, dados por pares clave => valor). En el archivo ej12.php se describe el funcionamiento con mayor detalle.

Instrucciones básicas de PHP

Operadores y expresiones

Gran parte de la sintaxis les resultará familiar tras haber visto JS:

- En la página oficial de PHP podemos encontrar un manual básico sobre el lenguaje de programación, incluyendo información sobre [expresiones](#) y sobre [operadores](#).
- La mayor parte de operadores aritméticos, de asignación, de comparación, etc. son iguales a los de lenguajes como JS.
- Por defecto, puede tratar de emplear los operadores que ya conoce, y si recibe un error de sintaxis revisar los ejemplos de clase o directamente la documentación del lenguaje.

Instrucciones básicas de PHP

Control de flujo

En el mismo manual mencionado encontramos información sobre las **estructuras de control**. Gran parte de la sintaxis les resultará familiar tras haber visto JS:

- La estructura **if** - **else** y anidados es igual que en JavaScript.
- Lo mismo ocurre con **switch**.
- Las instrucciones **while** y **do ... while** también funcionan igual.
- La versión clásica del **for** con sus tres secciones también es equivalente.

Ejemplo if-else

```
<?php
$dia = date("d");
if ($dia <= 15) {
    echo "Sitio activo";
} else {
    echo "Sitio fuera de servicio";
}
?>
```

Un primer ejemplo (ej13.php) de estructura **if ... else** sencillo, que como vemos se trata igual que en JS.

Ejemplo if-else y rand

```
<?php
$num = rand(1, 100);
echo $num;
echo "<br>";
if ($num <= 50) {
    echo "El número es menor o igual a 50.";
} else {
    echo "El número es mayor a 50.";
}
?>
```

- Aquí en ej14.php se usa la función `rand` para obtener un nº aleatorio entre un rango de enteros `$num=rand(1,100);`
- En la variable `$num` se almacena el entero generado.
- Podemos encontrar más información sobre `rand` [aquí](#).

Funciones con variables locales

Definición y llamada a funciones

Podemos crear funciones usando **function**, como en JS, y llamarlas pasándole parámetros (como **\$pers**):

```
<?php
function prueba($pers)
{
    $x = 5;
    echo "<p>Hola $pers, x en la funci&oacute;n" .
        " toma el valor: $x</p>";
}
prueba("Elena");
echo "<p>Fuera toma el valor : $x</p>";
?>
```

Como vemos en este ejemplo (ej15.php), se emplea en la función prueba una variable local **\$x**.

Funciones con variables locales

Devolución de valores con return

Como en JS y otros lenguajes, podemos devolver valores desde una función, haciendo uso de la palabra reservada **return** expresion;

```
<?php
function devuelveCubo($x)
{
    return $x*3;
}
$a = 7;
$b = devuelveCubo($a);
echo "<h1>El cubo de $a es $b</h1>";
?>
```

En este otro ejemplo (ej16.php) usamos la instrucción **return** para devolver el valor dado por la ejecución de la expresión indicada. Aquí vemos también la declaración de una variable local, **\$a**, cuyo valor se le pasa como argumento en la llamada a la función **devuelveCubo**, siendo posteriormente el resultado devuelto por la función almacenado en la otra variable global **\$b**.

Funciones con variables globales

A diferencia del ejemplo anterior, donde las variables globales se emplean para pasar un valor a una función o para almacenar su valor devuelto, en este ejemplo (ej17.php) no pasamos argumento a la función, pero esta tiene acceso a los valores de las variables al indicar dentro de la función ese aspecto:

```
global $x, $y;
```

```
<?php
```

```
    $x = 5;
```

```
    $y = 10;
```

```
    function Prueba()
```

```
    {
```

```
        global $x, $y;
```

```
        $y = $x + $y;
```

```
    }
```

```
    Prueba();
```

```
    echo "<p> El valor obtenido es $y</p>";
```

```
?>
```

Instrucciones iterativas: for simple

El siguiente ejemplo (ej18.php) genera una lista con los cuadrados de los números naturales del 0 al 10.

```
<ul>
  <?php
    for ($i = 0; $i <= 10; $i++) {
      print "<li>El cuadrado de $i es " . ($i*$i) . "</li>";
    }
  ?>
</ul>
```

Como podemos observar, la sintaxis es como la que hemos visto en JS.

Instrucciones iterativas: for simple

El siguiente ejemplo (ej19.php) emplea una notación alternativa para la definición de funciones, que luego podemos usar en expresiones interpoladas en cadenas.

```
<?php
$cuadrado = function ($n) {
    return pow($n, 2);
}
?>
<ul>
    <?php
        for ($i = 0; $i < 10; $i++) {
            print "<li>El cuadrado de $i es {$cuadrado($i)}</li>";
        }
    ?>
</ul>
```

Recorrido de arrays (ej20.php)

```
<?php $coches = array("Volvo", "BMW", "Toyota"); ?>
<h2>Array recorrido con for</h2>
<?php
    for ($i=0; $i<sizeof($coches); $i++) {
        print "<p>Coche $i: $coches[$i]</p>";
    }
?>
<h2>Array recorrido con foreach ("for-of" de JS)</h2>
<?php
    foreach ($coches as $coche) {
        print "<p>$coche</p>";
    }
?>
<h2>Array recorrido con foreach con clave-valor</h2>
<?php
    foreach ($coches as $i => $coche) {
        print "<p>Coche $i: $coche</p>";
    }
?>
```

PHP como lenguaje de servidor web

- Como hemos visto, PHP cuenta con los mecanismos básicos de cualquier lenguaje de programación generalista.
- No obstante, su papel principal es como servidor web, esperando peticiones de los clientes y devolviendo resultados, una vez accedido a los servicios necesarios presentes en el servidor (archivos, bases de datos, etc.)
- Habitualmente recibirá las peticiones mediante:
 - El `action` de un formulario HTML.
 - Peticiones asíncronas desde un lenguaje de cliente (JS), que canalice la comunicación cliente-servidor, preparando datos que enviar al servidor y posteriormente procesando la respuesta recibida del mismo.
- El servidor recibirá las peticiones, con sus datos asociados e info. sobre el estado del servidor, entre otras cosas. Dispondrá, entre otras cosas, de variables *superglobales* predefinidas (más detalle [aquí](#)).

Variables predefinidas principales

- `$_GET`: variables recibidas de la petición de tipo GET.
- `$_POST`: variables recibidas de la petición de tipo POST.
- `$_REQUEST`: variables recibidas de cualquier tipo de petición.
- `$_FILES`: variables procedentes de la carga de ficheros en un formulario.
- `$_SESSION`: variables de sesión (conexión entre un determinado cliente y el servidor, hasta el cierre de la misma o del navegador).
- `$_SERVER`: variables relacionadas con la configuración y el estado del servidor.
- `$_ENV`: variables de entorno del servidor web.

Formularios HTML

- Un formulario permite que el visitante al sitio cargue datos y sean enviados al servidor.
- En la versión más primitiva, llamaremos a PHP directamente desde algún botón de *submit* del formulario, teniendo este las dos propiedades (action y method) que recordamos:

```
<form action="registrardatos.php" method="post">
```

- action toma el valor del fichero php a llamar (registrardatos.php).
- method tomará el valor post o get:
 - Normalmente un formulario se envía mediante post (los datos se envían en el cuerpo del formulario).
 - Si se emplea get, los datos se envían en la propia url de la página. Con este método estamos limitados en la cantidad de datos a enviar.

Formularios HTML

- Ahora veamos el cuadro de texto donde se escribe el nombre:

```
<label for="nom">Escriba su nombre:</label>
```

```
<input type="text" name="nombre" size="20" id="nom">
```

- El identificador nombre indicado en el atributo name permitirá a PHP recuperar lo que haya escrito el usuario.
- Seguidamente tenemos un botón

```
<input type="submit" value="Enviar">
```

para el envío de datos al servidor.

Formulario HTML y servidor PHP

Desde nuestro formulario HTML, dado aquí:

```
<form action="registrardatos.php" method="post">
    Escriba su nombre:
    <input type="text" name="nombre" size="20">
    <br>
    <input type="submit" value="Enviar">
</form>
```

enviamos la petición a registrardatos.php, que lo procesa:

```
<body>
    Hola <?php echo $_POST["nombre"]; ?><br>
    Ahora conozco tu nombre.
</body>
```

Usamos `$_POST["nombre"]`; para recuperar el dato del usuario identificado con la etiqueta nombre.

PHP y MySQL

PHP 5 y posteriores se comunican con MySQL usando:

- MySQLi
- PDO (PHP Data Objects)

Nosotros usaremos el protocolo PDO.

Nota: existe una tercera opción para conectar con MySQL, la librería de `php mysql`, pero se desaconseja su uso, entre otros, por motivos de seguridad. Una excepción a esto es si debemos usar versiones anteriores a PHP5, pues en las mismas no tenemos disponibles las dos opciones sugeridas.

PHP y MySQL

El objetivo es tener una base de datos MySQL en un servidor, y poder acceder a ella (para consultar y/o manipular datos) desde otro ordenador mediante un navegador.

- Para ello necesitamos conocer:
 - La **base de datos** a la que queremos hacer conexión,
 - un nombre de **usuario** con acceso a esa base de datos, y
 - su **palabra clave** (*password*).
- Además necesitamos crear un formulario HTML que permita al usuario (ajeno a MySQL) proporcionar los datos.
- PHP se encargará de enlazar MySQL con HTML.

PHP y MySQL

Para empezar, crearemos la base de datos prueba en MySQL e indicaremos que vamos a trabajar sobre ella.

```
CREATE DATABASE prueba
USE prueba
```

A continuación crearemos una tabla contactos con dos atributos: nombre y apellidos

```
CREATE TABLE contactos (
    nombre varchar(20),
    apellidos varchar(20),
    PRIMARY KEY (nombre, apellidos)
);
```

Por otra parte, necesitamos una página HTML con un formulario para recoger los datos:

```
<form action="inserta1.php" method="post">  
  Escriba su nombre:  
  <input type="text" name="nombre" size="20" />  
  <br />  
  Escriba sus apellidos:  
  <input type="text" name="apellidos" size="20" />  
  <br />  
  <input type="submit" value="Enviar" />  
</form>
```

PHP y MySQL

En enlace entre PHP y MySQL lo hace el fichero `inserta1.php`. Creamos un fichero con ese nombre y recuperamos la información introducida por el usuario mediante las etiquetas `nombre` y `apellidos`:

```
$nom = $_POST['nombre'];  
$ape = $_POST['apellidos'];
```


PHP y MySQL

Creamos tres variables y guardamos en ella la información de la base de datos, del usuario y de la contraseña de acceso a Mysql:

```
$db = 'mysql:host=localhost;dbname=prueba;charset=utf8';  
$user = 'root';  
$pass = '';
```

A continuación, creamos la conexión mediante el protocolo PDO:

```
$conn = new PDO($db,$user,$pass);
```

PHP y MySQL

Consultas preparadas

A continuación, escribimos la sentencia MySQL que queremos ejecutar y la guardamos en la variable `$sql`, por ejemplo:

```
$sql = "INSERT INTO contactos VALUES (:n,:a)";
```

Como todavía no conocemos los valores que debemos insertar (serán proporcionados por el usuario), escribimos marcadores de parámetros, en este caso `:n` y `:a`. La sentencia SQL puede contener cero o más marcadores de parámetros por los cuales los valores reales serán sustituidos cuando la sentencia sea ejecutada. Esto se denomina una consulta preparada, y evita los problemas de seguridad de otras alternativas que no contengan este tipo de marcadores de parámetros.

PHP y MySQL

Consultas preparadas

Lo siguiente es enlazar la consulta que hemos creado con la conexión a la base de datos. Para ello creamos una variable `$consulta` y usamos la función `prepare`. La sintaxis es la siguiente

```
$consulta = $conn->prepare($sql);
```

Puedes encontrar más información sobre la función `prepare` en

<http://php.net/manual/es/pdo.prepare.php>

PHP y MySQL

Para terminar, solo queda enlazar los marcadores `:n` y `:a` con los datos proporcionados por el usuario, que habremos guardado en las variables `$nom` y `$ape`, validando su tipo:

```
$consulta->bindParam(':n',$nom,PDO::PARAM_STR);  
$consulta->bindParam(':a',$ape,PDO::PARAM_STR);
```

Finalmente, ejecutamos la consulta:

```
$consulta->execute();
```

Es muy importante cerrar la conexión cuando se ha realizado la consulta. Esto se puede hacer con

```
$conn = null;
```

PHP y MySQL

El fichero inserta1.php queda por tanto:

<?php

```
header ('Content-type: text/html; charset=utf-8');
$nom = $_POST['nombre'];
$ape = $_POST['apellidos'];
$db = 'mysql:host=localhost;dbname=prueba;charset=utf8';
$user = 'root';
$pass = '';

$conn = new PDO($db,$user,$pass);
$sql = "INSERT INTO contactos VALUES (:n,:a)";
$consulta = $conn->prepare($sql);
$consulta->bindParam(':n',$nom,PDO::PARAM_STR);
$consulta->bindParam(':a',$ape,PDO::PARAM_STR);
$consulta->execute();

$conn = null;
echo "<strong> Dato insertado</strong>&nbsp;";
```

Consultas

- Además de insertar datos, también podemos consultar la base de datos.
- En general, podemos hacer cualquier operación de modificación de la base de datos cambiando la sentencia SQL que enviamos a través de PHP.
- Aunque no lo vemos en la asignatura, insistir en los protocolos de seguridad asociados al uso de PHP.
- Podemos tener este formulario para realizar la petición:

```
<form action="consulta1.php" method="post">  
  <input type="submit" value="Consultar BD" />  
</form>
```

, procesado en php como vemos en la diapositiva siguiente.

Consultas

```
<?php
    header ('Content-type: text/html; charset=utf-8');

    $db = 'mysql:host=localhost;dbname=prueba;charset=utf8';
    $user = 'root';
    $pass = '';

    $conn = new PDO($db,$user,$pass);
    $sql = "SELECT * FROM contactos";
    foreach ($conn->query($sql) as $fila) {
        echo "<strong>",$fila['nombre'], "</strong>&nbsp;";
        echo "<em>",$fila['apellidos'], "</em><br>";
    }

    $conn = null;
    echo "<br><strong>
Consulta realizada</strong>&nbsp;";
```

Para saber más

- <http://php.net/manual/es/index.php>
- <http://www.desarrolloweb.com/manuales/12/>
- <http://www.w3schools.com/php/>
- <http://www.tutorialesprogramacionya.com/phpya/>

Nota: estos son solamente algunos recursos útiles, pero se proporcionará mucho más material a través del sitio web de la asignatura. *Además, otros temas introducirán conceptos adicionales, como el uso de los principios de AJAX, que cambiará la forma en que se envían las peticiones o se plasman los resultados devueltos por PHP en las páginas llamantes.*

Llamando al servidor desde JS

La API Fetch

- Apostamos por aplicaciones SPA
- En lugar del action de un form, la llamada al servidor PHP la hacemos desde JS
- La forma nativa de hacerlo es mediante la [API Fetch](#)
- Vemos a continuación ejemplos de llamada desde JS y tratamiento en PHP

Llamando al servidor desde JS

Ejemplo - Llamada GET sin enviar datos

```
let url = new URL("http://localhost/bd2022/php/consulta1.php");
fetch(url)
.then(function (response) {
    if (response.ok) {
        return response.text();
    }
    return Promise.reject(response);
})
.then((text) => {
    const sal = document.getElementById("salida");
    sal.innerHTML = "<br>Peticion GET solicitada<br>" + text;
    return text;
})
.catch(function (error) {
    console.warn("Algo fue mal:", error);
});
```

Llamando al servidor desde JS

Ejemplo - Llamada GET sin enviar datos

- La función `fetch` hace la llamada a la URL indicada
- Una vez recibida la respuesta del servidor (`response`), comprobamos el estado de la misma:
 - Si fue bien, se devuelve el texto recibido.
 - En otro caso, se devuelve un error.
- Con el texto recibido, actualizamos el DOM.
- La función `then` espera la respuesta de una operación asíncrona. Los posibles errores se capturan con `catch`.
- Para cambiar a petición POST, basta con añadir un objeto a `fetch`: `fetch(url, { method: "POST" })`
- El código JS lo incluimos en alguna función y la asociamos a un evento como manejador del mismo.

Llamando al servidor desde JS

Ejemplo - Generalización

```
function peticion(metodo) {  
  let url = new URL("http://localhost/bd2022/php/consulta1.php");  
  fetch(url, { method: metodo })  
  .then(function (response) {  
    if (response.ok)  
      return response.text();  
    return Promise.reject(response);  
  })  
  .then((text) => {  
    const sal = document.getElementById("salida");  
    sal.innerHTML = `  })  
  .catch(function (error) {  
    console.warn("Algo ha ido mal: ", error);  
  });  
}  
  
const btnGet = document.getElementById("btnGet");  
btnGet.addEventListener("click", () => peticion("GET"));  
const btnPost = document.getElementById("btnPost");  
btnPost.addEventListener("click", () => peticion("POST"));
```

Enviando datos al servidor desde JS

Preparando la petición

```
const params = {
  nombre: document.getElementById("nombre").value,
  apellidos: document.getElementById("apellidos").value,
};
const requestObj = {
  method: metodo,
};

let url = new URL("http://localhost/bd2022/php/inserta2.php");
if (metodo == "GET") {
  url.searchParams.append("nombre", params.nombre);
  url.searchParams.append("apellidos", params.apellidos);
} else {
  const data = new FormData(document.getElementById("miForm"));
  requestObj["body"] = data;
}
```

Enviando datos al servidor desde JS

Preparando la petición

- Para enviar datos, requerimos cierta preparación de los datos.
- Diferirá dependiendo del método (GET o POST):
 - Para una petición GET, añadimos los pares clave-valor a la url, lo que podemos hacer a mano o actuando sobre `url.searchParams`.
 - Podemos generalizar el paso de nuestro objeto `params` a `searchParams`:

```
Object.keys(params).forEach((key) =>  
    url.searchParams.append(key, params[key])  
);
```

Enviando datos al servidor desde JS

Preparando la petición

- Para una petición POST, actualizamos el objeto pasado a `fetch`, incluyendo nuestros datos en el `body`.
- Como alternativa a `FormData`, podríamos pasar los datos como JSON:

```
requestObj["body"] = JSON.stringify(params);  
requestObj["Content-type"] = "application/json; charset=UTF-8";
```

- En ese caso, hay que modificar el PHP para acceder a los datos JSON recibidos:

```
if (count($_REQUEST) == 0) {  
    $_REQUEST = json_decode(file_get_contents('php://input'), TRUE);  
}
```

Enviando respuesta JSON desde servidor PHP

```
header ('Content-type: application/json');

$db = 'mysql:host=localhost;dbname=prueba;charset=utf8';
$user = 'root';
$pass = '';

$conn = new PDO($db,$user,$pass);
$sql = "SELECT * FROM contactos";
$q = $conn->query($sql);
$res = $q->fetchAll();

echo json_encode($res);

$conn = null;
```


Enviando respuesta JSON desde servidor PHP

Para poder enviar una respuesta JSON en lugar de texto o HTML, se deben incorporar varios aspectos:

- La cabecera
`header('Content-type: application/json');` para indicar el tipo de contenido a devolver.
- La preparación de los datos recibidos de la consulta:
`$q = $conn->query($sql);`
`$res = $q->fetchAll();`
, primero fijando la consulta `$q` y luego devolviendo los resultados en `$res`.
- La codificación de la respuesta con `json_encode`.
- El envío a la salida de lo anterior usando `echo`.

Recibiendo y procesando respuesta JSON en cliente JS

```
let url = new URL("http://localhost/bd2022/php/consulta2.php");
fetch(url, { method: metodo })
.then(function (response) {
    if (response.ok)
        return response.json();
    return Promise.reject(response);
})
.then((contactos) => {
    const sal = document.getElementById("salida");
    const t1 = document.createElement("p");
    t1.textContent = `Petición ${metodo} solicitada`;
    sal.innerHTML = "";
    sal.appendChild(t1);
    const ul = document.createElement("ul");
    for (let contacto of contactos) {
        const li = document.createElement("li");
        li.innerHTML = `${contacto.nombre}</strong> <em>${contacto.apellidos}</em>`;
        ul.appendChild(li);
    }
    sal.appendChild(ul);
})
.catch(function (error) {
    console.warn("Algo ha ido mal: ", error);
});
```

Recibiendo y procesando respuesta JSON en cliente JS

En el cliente, los cambios más relevantes son los siguientes:

- Al recibir la respuesta del servidor, en lugar de `text` accedemos a `response.json()`
- Al procesar esa estructura, ya tenemos en cuenta que:
 - Hemos recibido un array de objetos json, con lo que podemos recorrerlo como hemos visto en el tema anterior.
 - Podemos ir creando los elementos HTML que necesitemos en el DOM.
- Téngase en cuenta que el objeto recibido fue una lista de filas devueltas por la consulta, pero el dato recibido podría tratarse de cualquier otro objeto o array JSON, generado empleando los arrays posicionales y asociativos que se requiera en PHP.

Sintaxis alternativa de consulta preparada

Dentro de los ejemplos disponibles en el [tutorial sobre consultas preparadas](#) de la web de PHP, tenemos una alternativa a la sintaxis que hemos visto anteriormente:

```
$sql = "SELECT * FROM CONTACTOS WHERE nombre LIKE ? OR apellidos LIKE ?";  
$stmt = $conn->prepare($sql);  
$stmt->execute(["%{$bus}%","%{$bus}%"]);  
$data = $stmt->fetchAll(PDO::FETCH_ASSOC);
```

Como vemos, la alternativa a la sintaxis de parámetros nombrados como `:nombre` y `:apellidos` y las correspondientes funciones de tipo `bindParam` y `bindValue` para asociarles valores, se pueden indicar las posiciones de los parámetros con el carácter `?` y posteriormente pasarle los valores en un array incluido como argumento de la función `execute`.